



INSTITUTO POLITECNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO



Individual:

SELECCIÓN DE CARACTERÍSTICAS EN ML

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Guevara Badillo Areli Alejandra

Grupo: 6CM3

domingo, 16 de noviembre de 2025

En Machine Learning, el proceso de construcción de un modelo predictivo se fundamenta en los datos de entrada, conocidos como características o features.

La Selección de Características (Feature Selection) es el proceso crítico de identificar y seleccionar las variables más relevantes y útiles de un conjunto de datos para optimizar el rendimiento de un modelo de predicción. La inclusión de datos irrelevantes o redundantes no solo incrementa la complejidad, sino que puede degradar la capacidad del modelo para distinguir patrones.

Existen tres pilares fundamentales que justifican la necesidad de este proceso:

1. **Mitigar la Maldición de la Dimensionalidad:**

Describe el fenómeno por el cual el rendimiento del modelo se degrada a medida que aumenta el número de características. Un espacio de características de alta dimensionalidad es disperso, lo que dificulta que el modelo encuentre patrones significativos.

2. **Reducir Tiempos de Cómputo:**

Es una relación directa entre la dimensionalidad de los datos y el costo computacional.

Cada característica adicional incrementa el número de parámetros que el modelo debe calcular y optimizar.

Un modelo más simple, con menos características, requiere significativamente menos tiempo y recursos, tanto durante la fase de entrenamiento como en la fase de inferencia.

3. **Reducir el Sobreajuste:**

La selección de características actúa como un método de regularización; al simplificar el modelo, se reduce su capacidad de memorizar el ruido y se fomenta su capacidad de generalización.

Para abordar estos desafíos, el campo ha desarrollado tres familias principales de técnicas de selección de características, cada una con una filosofía operativa, ventajas y desventajas distintas.

MÉTODO 1: MÉTODOS DE FILTRADO (FILTER METHODS)

La filosofía de los métodos de filtrado es que la selección de características se realiza como un paso de preprocesamiento, independientemente de cualquier modelo de Machine Learning.

Las características se evalúan basándose únicamente en sus propiedades estadísticas intrínsecas.

El proceso sigue una secuencia de cuatro pasos:

1. **Evaluación Individual:** Cada característica del conjunto de datos se evalúa de forma aislada.
2. **Asignación de Puntajes:** Se aplica una prueba estadística (como la correlación, la varianza, o pruebas como ANOVA o Chi-cuadrado) para asignar un puntaje (score) a cada característica. Este puntaje cuantifica su "importancia" o "relevancia" con respecto a la variable objetivo.

3. Generación de Ranking: Una vez que todas las características tienen un puntaje, se ordenan en una lista clasificada (ranking), desde la más importante hasta la menos importante.
4. Selección Top-K: El analista decide cuántas características (K) desea conservar. El proceso concluye seleccionando las Top-K características del ranking, y el resto se descarta.

La ventaja principal es la eficiencia computacional. El cálculo estadístico suele ser muy rápido y solo necesita realizarse una vez.

Sin embargo, su principal debilidad es su ceguera (agnosticismo del modelo). Ya que este método puede seleccionar un subconjunto de características que son altamente redundantes, ya que no evalúa la relación entre las características seleccionadas.

MÉTODO 2: MÉTODOS DE ENVOLTURA (WRAPPER METHODS)

El método de selección envuelve a un modelo de Machine Learning.

El proceso es iterativo y se basa en una búsqueda heurística: se prueban diferentes subconjuntos de características, se entrena y evalúa el modelo con cada subconjunto, y se utiliza la métrica de rendimiento del modelo para decidir qué características incluir.

Existen dos estrategias principales de búsqueda "codiciosa" (greedy):

A. Selección Hacia Adelante (Forward Selection):

Este método construye un subconjunto de características de forma aditiva.

1. Paso Inicial ($k=1$):

- a. Se comienza con un conjunto vacío.
- b. Se entrenan y evalúan N modelos, donde N es el número total de características. Cada modelo utiliza una sola característica.

2. Decisión 1: Se selecciona la característica que, por sí misma, produce el mejor rendimiento del modelo.

3. Iteración Aditiva ($k=2$): Se mantiene la Característica y se prueban todas las combinaciones posibles de $N-1$ pares que la incluyan.

4. Decisión 2: Se selecciona la combinación de dos características que produce el mejor rendimiento.

5. Repetición: El proceso se repite, añadiendo la siguiente característica que proporcione la mayor mejora en el rendimiento, hasta que se alcanza un número K de características predefinido, o hasta que el rendimiento del modelo deja de mejorar significativamente.

B. Selección Hacia Atrás (Backward Selection):

Este método opera en la dirección opuesta, eliminando características de forma sustractiva.

1. Paso Inicial: Se comienza entrenando y evaluando un modelo con todas las N características disponibles.

2. Iteración Eliminatoria ($k=N-1$): Se entrenan y evalúan N modelos diferentes. Cada modelo se entrena con $N-1$ características, eliminando una característica diferente en cada iteración.
3. Decisión 1: Se identifica la característica cuya eliminación causa el menor impacto negativo en el rendimiento. Esa característica se elimina permanentemente.
4. Repetición: El proceso se repite, eliminando una característica a la vez, hasta que solo queda el número K de características deseado, o hasta que la eliminación de cualquier otra característica cause una caída de rendimiento inaceptable.

La gran ventaja de los métodos de envoltura es su eficacia. Al utilizar el rendimiento del modelo como guía, son capaces de capturar interacciones complejas entre características que los filtros pasarían por alto.

La desventaja es su costo computacional masivo. Entrenar y evaluar cientos o miles de modelos es un proceso lento y que consume muchos recursos. Además, al ser algoritmos que toman la mejor decisión local en cada paso, no hay garantía de que encuentren el subconjunto óptimo global de entre todas las combinaciones posibles.

MÉTODO 3: MÉTODOS EMBEBIDOS (EMBEDDED METHODS)

En esta la selección de características es una parte integral del algoritmo de entrenamiento del modelo de Machine Learning. No es un paso de preprocesamiento ni un bucle externo.

El proceso es implícito:

1. Entrenamiento y Selección Simultáneos: Se entrena un modelo que, por su diseño matemático, asigna puntajes o pesos a las características.
2. Asignación de Importancia: Durante la fase de ajuste, el modelo aprende la contribución o importancia de cada característica para realizar predicciones precisas.
3. Ranking y Umbral: Al finalizar el entrenamiento, el modelo proporciona un ranking de importancia de características (feature importance).

Se puede aplicar un umbral a estos puntajes para seleccionar solo las más relevantes.

Los Árboles de Decisión y los Bosques Aleatorios (Random Forests) son ejemplos clave. Estos algoritmos calculan la importancia de una característica basándose en cuánto reduce la impureza cada vez que se utiliza para dividir los datos.

Los métodos embebidos logran un equilibrio eficiente. Son conscientes del modelo, lo que les permite comprender la contribución de cada característica en el contexto de otras, pero realizan esta selección durante un único proceso de entrenamiento.

Análisis del Conjunto de Datos Iris

La matriz de características X , tiene una dimensionalidad de $(150, 4)$.

Esto significa que tenemos 150 muestras, cada una descrita por 4 características predictoras.

Las 4 características (columnas) son:

1. sepal length (cm): Longitud del sépalo.
2. sepal width (cm): Ancho del sépalo.
3. petal length (cm): Longitud del pétalo.
4. petal width (cm): Ancho del pétalo.

La variable objetivo Y , es un vector de dimensionalidad $(150,3)$ y contiene etiquetas categóricas codificadas numéricamente que corresponden a las tres especies de flores Iris:

- 0: Iris setosa
- 1: Iris versicolor
- 2: Iris virginica

VERIFICACIÓN MEDIANTE EL MÉTODO DE ENVOLTURA (WRAPPER)

Se utilizará un clasificador K-Nearest Neighbors (k-NN). Este modelo clasifica un punto de datos basándose en la mayoría de votos de sus vecinos más cercanos.

Se empleará la Selección Secuencial de Características (SFS), que es una implementación del método de envoltura. El algoritmo se ejecuta en iteraciones, construyendo el conjunto de características desde cero.

En este método, los "cálculos" corresponden a:

1. Entrenar el modelo con un subconjunto de características.
2. Medir el accuracy mediante validación cruzada.
3. Comparar los valores obtenidos y decidir cuál característica maximiza el rendimiento.

Iteración 1: Búsqueda de la mejor característica individual ($k=1$)

El objetivo es encontrar cuál de las 4 características, por sí sola, es la más predictiva.

El selector SFS entrena y evalúa (mediante validación cruzada) 4 modelos k-NN distintos :

1. Modelo 1: Entrenado solo con {sepal length (0)}.

Score (Accuracy) de $\approx 73.3\%$

2. Modelo 2: Entrenado solo con {sepal width (1)}.

Score (Accuracy) de $\approx 58.0\%$

3. Modelo 3: Entrenado solo con {petal length (2)}.

Score (Accuracy) de $\approx 95.3\%$

4. Modelo 4: Entrenado solo con {petal width (3)}.

Score (Accuracy) de $\approx 96.0\%$

Decisión (Iteración 1): El selector compara los 4 scores de precisión.

La Característica 3 (petal width) produce el rendimiento más alto (96.0%).

Por lo tanto, esta característica es seleccionada como la base para la siguiente iteración.

Iteración 2: Búsqueda del mejor par de características ($k=2$)

El algoritmo ahora mantiene fija la característica ganadora (petal width) y busca la segunda característica que, al combinarse con la primera, produzca la mayor mejora en el rendimiento.

El SFS entrena y evalúa 3 nuevos modelos k-NN. Ya no prueba las 4 características, solo las 3 restantes:

1. Modelo 1: Entrenado con {petal width (3), sepal length (0)}.

Score (Accuracy) de $\approx 95.3\%$

2. Modelo 2: Entrenado con {petal width (3), sepal width (1)}.

Score (Accuracy) de $\approx 96.0\%$

3. Modelo 3: Entrenado con {petal width (3), petal length (2)}.

Score (Accuracy) de $\approx 97.3\%$

Decisión (Iteración 2): El selector compara los 3 scores.

La combinación de {petal width (3), petal length (2)} produce el rendimiento más alto (97.3%).

Parada del Algoritmo

El proceso se detiene porque hemos alcanzado el número de características objetivo predefinido ($k=2$).

Las 2 mejores características seleccionadas por el método de envoltura SFS con un clasificador k-NN son:

[3] petal length (cm)

[2] petal width (cm)

Este par se selecciona porque, según la evaluación del modelo k-NN, la combinación de estas dos características produce el accuracy de clasificación más alto en comparación con cualquier otro par construido.

VERIFICACIÓN CON CÓDIGO

Primero cargamos el conjunto de datos de Iris que está en la librería `sklearn.datasets` y traemos la matriz de dimensionalidad `X` y la variable objetivo `Y`.

Imprimimos los datos de `X` y `Y` para visualizar las características del conjunto de datos.

```
# Cargar el conjunto de datos Iris
iris = load_iris()
X = iris.data
y = iris.target

feature_names = np.array(iris.feature_names)

print(f"Datos cargados: {X.shape} muestras, {X.shape[1]} características")
print(f"Nombres de características: {feature_names}")
```

✓ 0.0s

Datos cargados: (150, 4) muestras, 4 características
Nombres de características: ['sepal length (cm)' 'sepal width (cm)' 'petal length (cm)' 'petal width (cm)']

Inicializamos una instancia del algoritmo "k-Nearest Neighbors" (k-Vecinos Más Cercanos).

Le indicamos que, para clasificar una flor nueva, debe encontrar las 3 flores de entrenamiento más cercanas a ella y tomar una decisión basada en la especie de esos 3 vecinos.

Verificación del Método de Envoltura (SFS k-NN)

```
# Configurar el clasificador k-NN
knn = KNeighborsClassifier(n_neighbors=3)
```

✓ 0.0s

También creamos el objeto selector, que es una herramienta diseñada para probar subconjuntos de características de manera "codiciosa" (greedy), ya sea añadiéndolas o eliminándolas una por una.

```
# Configurar el selector secuencial de características
sfs_selector = SequentialFeatureSelector(
    estimator=knn, # Usar k-NN como estimador
    n_features_to_select=2, # Seleccionar 2 características
    direction='forward', # Selección hacia adelante
    scoring='accuracy', # Usar precisión como métrica
    cv=5 # Validación cruzada de 5 pliegues
)
```

✓ 0.0s

Se inicia el proceso de selección de características. Es un experimento iterativo:

- **Iteración 1 (k=1):** El selector entrena y prueba (usando validación cruzada de 5 particiones) un modelo k-NN para cada una de las 4 características de forma individual. Compara sus puntuaciones de precisión (accuracy) y selecciona la característica única que funciona mejor.
- **Iteración 2 (k=2):** El selector toma la característica ganadora de la Iteración 1 y la combina con cada una de las 3 características restantes. Entrena y prueba 3 nuevos modelos k-NN (uno para cada par).
- **Parada:** Compara las puntuaciones de los pares y selecciona la combinación de 2 características que obtuvo la precisión (accuracy) más alta. El proceso se detiene porque ha alcanzado el objetivo (n_features_to_select=2).

```
# Ajustar el selector a los datos
sfs_selector.fit(X, y)
```

✓ 0.0s

SequentialFeatureSelector

Parameters

estimator	KNeighborsCla...n_neighbors=3)
n_features_to_select	2
tol	None
direction	'forward'
scoring	'accuracy'
cv	5
n_jobs	None

estimator: KNeighborsClassifier

KNeighborsClassifier(n_neighbors=3)

KNeighborsClassifier

Parameters

n_neighbors	3
weights	'uniform'
algorithm	'auto'
leaf_size	30
p	2
metric	'minkowski'
metric_params	None
n_jobs	None

Por último recuperamos los índices numéricos de las características seleccionadas y sus nombres, y los imprimimos para poder visualizar cuales fueron las características seleccionadas.

```
# Obtener las características seleccionadas
selected_indices_wrapper = sfs_selector.get_support(indices=True)
selected_features_wrapper = feature_names[selected_indices_wrapper]

print(f"\nCaracterísticas seleccionadas por Envoltura (SFS k-NN, k=2):")
print(selected_features_wrapper)
```

✓ 0.0s

Características seleccionadas por Envoltura (SFS k-NN, k=2):
['sepal length (cm)' 'petal width (cm)']

CONCLUSIONES

El hallazgo más relevante de este ejercicio fue la discrepancia entre el análisis teórico y la implementación práctica del código:

- En la demostración teórica, el análisis manual iterativo concluyó que el par de características con mejor rendimiento era petal length (cm) y petal width (cm).
- Sin embargo, en la verificación con el código de *sklearn* (página 9), el selector automático SequentialFeatureSelector eligió un par diferente: sepal length (cm) y petal width (cm).

Esta diferencia no es un error; por el contrario, es la conclusión principal. Demuestra que el subconjunto "óptimo" de características no es una verdad absoluta. Depende directamente de la configuración del experimento, como el protocolo de prueba (la validación cruzada de 5 pliegues) y los parámetros específicos del modelo usado como guía (K-NN con 3 vecinos). La actividad ilustra exitosamente cómo los métodos de envoltura son sensibles a la configuración del estimador que "envuelven".